PS1-Report

在完成这些作业的过程中，我参考了多个技术网站和教程，解决了从语法细节到算法框架的各类难点。以下按作业顺序分别说明每个问题的解决思路、关键知识点以及参考来源。为清晰标注，正文中用方括号数字（如[1]）表示引用，并在每段末尾附上对应网页链接。

一、Flowchart

实现时需要注意流程图中某些分支没有输出操作，需要用 pass 占位。此外，输入处理采用了 map(float, int_val.split(','))的方式，将逗号分隔的字符串转换为浮点数并解包给三个变量。

在解决输入解析和条件分支逻辑时，我参考 PingCode 上关于 Python 输入处理的文章[1]，其中关于字符串分割和类型转换的示例帮助我理解了 split()与 map()的配合使用。

（参考：[1] https://docs.pingcode.com/ask/1120526.html）

二、Matrix multiplication

在创建矩阵时，我参考了 PingCode 上关于 NumPy 矩阵创建的文章[2]，学会了使用 np.random.randint(0, 51, size=(5, 10))生成指定范围的随机整数矩阵。对于矩阵乘法的三重循环实现，腾讯云开发者社区的一篇文章[3]提供了清晰的示例，帮助我理解了循环变量的范围和累加方式。此外，PingCode 上关于提取矩阵列元素的文章[4]让我明白了如何通过切片操作获取矩阵的列，这在内层循环中非常关键。遇到的主要难点是结果矩阵的初始化，最初使用了 np.zeros 但得到的是浮点数，后来通过指定 dtype=int 解决了问题。

（参考：[2] https://docs.pingcode.com/ask/ask-ask/941573.html；[3] https://cloud.tencent.com/developer/article/2455147；[4] https://docs.pingcode.com/ask/1139875.html）

三、Pascal triangle

在理解递推关系和 for 循环实现时，菜鸟教程上关于 Python 列表操作的示例[5]给了我启发。同时，廖雪峰 Python 教程中关于 for 循环和列表的讲解[6]帮助我理清了逐行生成的循环结构。遇到的主要难点是变量作用域和列表更新，通过将 Row 和 New_Row 在循环中正确交替赋值，最终实现了正确的递推。

（参考：[5] https://liaoxuefeng.com/books/python/introduction/index.html；[6] https://www.liaoxuefeng.com/wiki/1016959663602400/1017105145133280）

四、Add or double

这个问题解决的关键是 Agent 提示我，将数字转换为二进制后，设二进制位数为 B，其中 1 的个数为 C，则最少步数等于 B + C - 2。这个公式的推导关键在于：从最高位到最低位每多一位需要一次翻倍（B-1 次），除最高位外每个 1 需要一次加 1 操作（C-1 次），合计 B+C-2。

在理解二进制转换和位运算时，问题得到快速解决。PingCode 上关于 Python 二进制转换的文章[7]提供了 bin()函数的详细用法和手动转换的示例，帮助我快速实现了二进制字符串的获取和 1 的计数。

（参考：[7] https://docs.pingcode.com/ask/1120526.html）

五、Dynamic programming

本作业要求在数字 123456789 之间插入+或-，或将数字连接，使表达式的值等于目标整数。5.1 要求打印所有等于 k 的表达式，5.2 要求统计 1 到 100 每个整数的解数量并绘图找出最大/最小解对应的整数。

解决思路是使用深度优先搜索（DFS）递归穷举所有可能。数字之间有 8 个空隙，每个空隙有 3 种选择（加、减、合并），总共 3^8=6561 种表达式。递归函数 dfs(idx, output)中，idx 表示当前处理到第几个数字，output 是当前拼好的表达式字符串。终止条件是 idx==8，此时用 eval()计算表达式的值并与目标比较。5.2 中通过 Count_solutions(k)统计每个 k 的解数量，再用列表推导式得到 Total_solutions，最后用 enumerate 找出最大值和最小值对应的整数。

在实现递归回溯时，labuladong 的回溯算法框架文章[8]给了我极大的帮助。它清晰地阐述了回溯算法的三要素：路径（已做的选择）、选择列表（当前可以做的选择）、结束条件。我将其对应到本题中：路径就是 output 字符串，选择列表就是三种运算符，结束条件就是处理完所有数字。此外，B 站上的算法视频教程[2]让我更直观地看到了递归树的展开过程。遇到的主要难点是 nonlocal 关键字的使用——在嵌套函数中修改外层变量 Visited 时必须声明 nonlocal，否则会报 UnboundLocalError。另外，eval()函数虽然方便，但要注意安全性，本题中表达式仅由数字和+、-组成，因此是安全的。

对于 5.2 的统计部分，最初我打算对每个 k 都调用一次 Count_solutions，这样会重复枚举 6561 种表达式 100 次，效率较低。后来参考了 labuladong 动态规划文章中关于“重叠子问题”的思路[9]，意识到可以只枚举一次所有表达式，用字典或列表记录每个结果出现的次数，从而大幅提高效率。虽然最终作业中仍使用了循环调用，但这个优化思路让我对动态规划与

回溯的关系有了更深的理解。

（参考：[8] https://labuladong.online/zh/algo/essential-technique/backtrack-framework/；[2] https://www.bilibili.com/video/BV1GdLiz1EhV/；[9] https://labuladong.online/zh/algo/essential-technique/dynamic-programming-framework/）

六、补充参考资源

除了上述已参考的网站，我还查阅了 Python 官方文档中关于函数定义和递归的内容[10]，以及 itertools 模块的文档[11]，特别是 itertools.product 可以用于生成所有运算符组合，虽然本题采用了递归实现，但 product 提供了一种更简洁的穷举思路。GeeksforGeeks 上的矩阵乘法教程[12]提供了三重循环的多种语言实现，帮助我验证了 Python 版本的逻辑。LeetCode 上的回溯算法题单[13]和动态规划题单[14]则让我看到了更多类似问题的变体，加深了对算法框架的理解。这些资源共同帮助我解决了从语法细节到算法设计的各类难点，使五个作业得以顺利完成。

（参考：[10] https://docs.python.org/zh-cn/3/tutorial/controlflow.html#defining-functions；[11] https://docs.python.org/zh-cn/3/library/itertools.html；[12] https://www.geeksforgeeks.org/c-program-multiply-two-matrices/；[13] https://leetcode.cn/tag/backtracking/；[14] https://leetcode.cn/tag/dynamic-programming/）